

퀄리저널 관리자 시스템 개발 완료 정의의 정합성 분석

1. 완료 정의 및 체크리스트 요약

"퀄리저널 관리자 시스템 개발 완료 정의" 보고서는 관리자 페이지(admin.standardai.co.kr)에서 뉴스 수집→편집→요약/번역→승인→보고서 생성→결과 내보내기→백업→발행까지의 전체 워크플로우가 원활하게 수행될 때를 개발 완료로 규정한다. 정의서에서는 다음과 같은 세부 기준을 제시한다:

- **프론트엔드(관리자 UI)**: 키워드 입력 후 뉴스 수집이 가능하고, 기사 리스트를 편집·검토하며 승인과 편집자 코멘트 입력을 지원하고, **15 개 이상 승인된 기사**가 있어야 발행 버튼이 활성화된다. 발행 시 성공/오류 상태를 UI에 표시하며 반응형 디자인과 로딩 아이콘 등 UX가 갖춰져야 한다.
- **백엔드/API**: 여러 뉴스 소스로부터 자동 수집하고 중복을 제거하는 API, AI 요약 및 번역 처리 API, 기사 승인 상태와 편집자 코멘트를 저장하는 API, 수집→재구성→선정→발행의 워크플로우 자동화, 최종 보고서(Markdown/HTML) 생성과 백업 기능이 제공되어야 한다.
- **배포 및 접근성**: 관리자 UI와 API가 **admin.standardai.co.kr** 도메인에 배포되어 접근 가능하고, Cloud Run 배포가 최신 코드로 유지되어야 한다.
- **자동화 및 모니터링**: 일일 발행을 위한 스케줄러, CI/CD 파이프라인, 백업·모니터링 자동화가 이루어져야 한다.

이상의 조건을 모두 만족할 때 개발 완료로 판단한다.

2. 첨부된 코드 및 자료의 기능 구현 현황

2.1 수집·선정·발행 파이프라인

- **뉴스 수집**: `engine_core.py`와 `orchestrator.py`는 공급(official) 소스와 커뮤니티 소스를 대상으로 크롤링을 수행하고, 중복 제거 및 스코어링 로직을 포함한다. `orchestrator.py`는 `--collect / --collect-community / --collect-keyword` 옵션을 제공하여 키워드별 수집도 지원한다. 이는 정의서의 수집 조건과 일치한다.
- **기사 선정 및 승인**: `server_quali.py`는 `/api/items/{item_id}/publish` 엔드포인트를 통해 개별 기사에 대해 승인 여부와 편집자 코멘트를 저장하고, `/api/status`를 호출하면 승인된 기사 수와 **게이트(gate_required)** 설정(기본값 15개)에 따라 발행 가능 여부를 알려 준다. 프론트엔드는 이 값을 바탕으로 발행 버튼을 활성화한다. 따라서 **최소 승인 15개** 조건이 코드로 구현되어 있다.
- **보고서 생성**: `/api/report` 엔드포인트와 `_generate_summary_md` 함수는 승인된 기사 목록을 이용해 Markdown 보고서를 생성하고 `archive/reports` 폴더에 저장한다. `export_md/csv` 엔드포인트를 통해 최종 선택본을 MD/CSV로 내보낼 수 있다. UI에서도 “보고서 생성” “MD/CSV 내보내기” 버튼이 구현되어 있다.

2.2 요약·번역 기능

- 엔진 코드에는 `_best_ko_summary`와 `_translate_en_to_ko` 함수가 존재하여 요약과 번역을 처리할 수 있도록 설계되어 있으나, 기본 `selected_articles.json`에는 `summary` 필드만 존재하고 `summary_ko` 또는 `desc` 필드가 대부분 빈 상태이다. `/api/enrich/keyword`와 `/api/enrich/selection` API는 현재 기사에 포함된 요약/번역 필드를 이용해 추가 Markdown 파일을 생성할 뿐, 수집 단계에서 자동으로 기사 객체에 번역 결과를 쓰지는 않는다. 외부 번역 API 키가 환경변수(`API_KEY` 또는 `QUALI_DISABLE_OPENAI`)에 따라 비활성화될 수 있어서 요약·번역이 항상 수행되는 구조는 아니다. 정의

서의 “요약/번역 기능이 구현되어야 한다”는 항목과 비교하면 번역 결과를 기사 데이터에 저장하고 UI에서 확인할 수 있도록 하는 부분이 부족하다.

2.3 배포 및 접근성

- `server_quali.py` 와 `index.html` 은 FastAPI 서버와 단일 HTML/JS 프론트엔드를 하나의 Docker 이미지로 빌드하도록 설계되어 있으며, GCP Cloud Run 배포를 위한 `Dockerfile` 과 `requirements.txt` 가 존재한다. 환경변수 `ADMIN_TOKEN` 등으로 인증을 관리한다.
- 그러나 커스텀 도메인(`admin.standardai.co.kr`) 설정은 아직 적용되지 않았다. 인수인계 보고서에서도 “현재 Cloud Run이 제공하는 기본 URL을 사용하므로 `standardai.co.kr` 도메인 연결을 설정해야 한다”는 다음 단계 작업으로 남아 있다.

2.4 자동화 및 백업

- 현재 `/api/tasks/flow` 를 통해 커뮤니티·키워드·데일리 플로우를 백그라운드에서 실행할 수 있지만, 정해진 시간에 자동으로 실행하는 스케줄러는 구현되어 있지 않다. 인수인계 보고서도 “일일 보고서 생성 기능을 Cloud Scheduler 등으로 자동화해야 한다”고 명시한다.
- 백업은 `server_quali.py` 의 `/api/backup/notify` 엔드포인트로 성공 여부를 기록하는 수준이며, 실제로 생성된 보고서를 Google Cloud Storage(GCS)나 다른 스토리지에 업로드하는 자동화는 존재하지 않는다. 인수인계 보고서는 현재 수동으로 실행되는 `run_backup_gcs.ps1` 스크립트를 CI 파이프라인 또는 크론 작업으로 자동화해야 한다고 제안하고 있다.

3. 정합성 평가 및 보완이 필요한 부분

1. **도메인 연결 미구현** - 정의서에서는 `admin.standardai.co.kr` 도메인에서 서비스가 제공되어야 한다고 명시하지만, 인수인계 자료에 따르면 현재는 Cloud Run 기본 URL로 서비스되고 있으며 도메인 매핑과 DNS 설정이 남아 있다. 따라서 도메인 매핑이 완료되지 않았으므로 정합성이 떨어진다.
2. **일일 발행 자동화 미구현** - 전체 워크플로우를 자동화하려면 수집→선정→발행을 일정에 맞춰 실행해야 하지만, 현재는 수동으로 `/api/report` 또는 `/api/tasks/flow` 를 호출해야 한다. Cloud Scheduler/cron 작업으로 일일 실행을 자동화해야 한다.
3. **백업 자동화 미구현** - 정의서에서는 결과물 저장 및 GCS 백업이 구현되어야 함을 강조하지만, 코드상으로는 `/api/backup/notify` 로 백업 완료를 기록하는 기능만 있고 실제 백업을 수행하는 스크립트는 수동으로 실행되고 있다. 백업 자동화가 필요하다.
4. **요약/번역 데이터 처리 불완전** - 엔진 코드에 요약과 번역 기능이 존재함에도 불구하고, `selected_articles.json` 등 실제 데이터에는 번역 결과가 저장되지 않는다. 관리자 UI에서도 기사별로 번역된 요약을 확인하거나 수정하는 기능이 제한적이다. 실제로 요약·번역이 동작하도록 외부 번역 API 키를 설정하고, 번역 결과를 기사 객체(`summary_ko` 등)에 저장해 UI에서 확인·수정할 수 있도록 해야 정의서의 요구를 충족한다.
5. **커스텀 도메인 기반 배포 테스트 미흡** - 정의서에서 반응형 디자인과 다양한 브라우저 호환성을 강조하지만, 실제 `index.html`은 PC 화면에 최적화되어 있으며 모바일/태블릿 테스트나 한국어 로케일 외 환경에 대한 검증이 부족하다. 또한 접근성 측면에서 로그인/토큰 관리 UI가 초기화되어 있지 않아 처음 사용하는 운영자가 혼란을 겪을 수 있다.
6. **운영자용 가이드와 DoD 사이의 용어 차이** - 정의서에서는 “선정본 승인: 최소 15개 기사를 선택해야 발행 가능”이라고 설명하지만, `index.html` UI에서는 게이트 슬라이더가 존재해 `gate_required` 값을 사용자

가 조정할 수 있다. 기본값은 15개이나, 운영자가 가치를 바꿀 수 있는 기능은 문서에 설명되어 있지 않아 문서와 시스템 사이에 약간의 차이가 있다. 가이드에 슬라이더 기능과 게이트 설정 API를 명시할 필요가 있다.

4. 보완 작업을 위한 개발자용 작업 지침

아래는 위 정합성 분석을 바탕으로 개발 완료를 위해 수행해야 할 주요 보완 작업을 **프론트엔드, 백엔드/API, 배포, 자동화/백업** 측면으로 정리한 작업 지침이다. 각 항목에는 필요성, 구체적 작업 내용, 참고 자료를 제공한다.

4.1 도메인 매핑 및 HTTPS 설정

- **왜 필요한가?** 정의서에 명시된 `admin.standardai.co.kr` 도메인에서 UI와 API를 제공해야 하며, 운영자가 익숙한 브랜드 도메인으로 접속할 수 있어야 한다. 현재 Cloud Run 기본 도메인으로 서비스되고 있어, SSL 인증서와 DNS 설정이 미완료 상태다.
- **할 일**
 - 가비아(또는 해당 도메인 등록업체)에서 `admin` 하위 도메인을 Cloud Run 서비스의 CNAME 값으로 설정한다. DNS TTL을 고려해 변경 후 최대 24 시간 정도의 전파 시간을 예상한다.
 - GCP Console 또는 `gcloud run domain-mappings create` 명령을 사용해 Cloud Run 서비스와 커스텀 도메인을 연결한다.
 - Cloud Run 설정에서 HTTPS 전용(redirect HTTP → HTTPS)과 TLS 인증서 자동 관리를 활성화한다.
 - 환경변수 `APPROVE_UI_URL` 등에서 도메인을 참조하는 값이 있다면 모두 `https://admin.standardai.co.kr`로 업데이트한다.
 - 도메인 매핑 이후, 브라우저에서 URL을 통해 관리자 UI가 정상적으로 로딩되고 API 호출이 성공하는지 테스트한다.

4.2 일일 발행 및 백업 스케줄러 구현

- **왜 필요한가?** 관리자는 매일 수동으로 보고서를 생성하거나 플로우를 실행해야 한다. 정의서에서는 스케줄러에 의해 **자동 수집·발행**이 실행되어야 하고, 인수인계 보고서 역시 Cloud Scheduler나 cron 기반 자동화를 권장한다.
- **할 일**
 - Google Cloud Scheduler를 사용해 매일 원하는 시간(예: 오전 9시) `POST https://admin.standardai.co.kr/api/tasks/flow` 또는 `/api/report` 엔드포인트를 호출하는 HTTP 크론 작업을 생성한다. 헤더에는 `Authorization: Bearer <ADMIN_TOKEN>`을 포함한다.
 - 커뮤니티 및 키워드 뉴스의 경우에는 `/api/tasks/flow` `kind` 파라미터(`daily` `community keyword`)를 적절히 설정하여 여러 작업을 순차적으로 실행하도록 한다.
 - 작업 로그는 Cloud Run Log Explorer에서 확인할 수 있으며, 실패할 경우 Slack 알림을 받을 수 있도록 Alert Policy를 생성한다.
 - 백업 자동화는 Cloud Scheduler 또는 GitHub Actions Cron workflow를 이용해 `gsutil cp` 명령으로 `archive/` 폴더의 보고서·선정보 파일을 GCS 버킷으로 복사하도록 구성한다. 현재 수동으로 실행되는 `run_backup_gcs.ps1` 스크립트의 동작을 파악하여 셸 스크립트(`bash` 또는 Python)로 재작성하는 것이 좋다. 백업 성공 후 `/api/backup/notify`를 호출하여 UI에서 상태를 확인할 수 있게 한다.

4.3 요약/번역 파이프라인 개선

- **왜 필요한가?** 정의서에서는 뉴스 카드를 요약·번역하는 기능을 필수로 요구하지만, 현재 구현에서는 번역 결과가 기사 데이터에 반영되지 않아 관리자나 독자가 한국어 요약을 확인할 수 없다. 또한 기사별 요약을 편집자가 수정할 수 있는 UI도 미흡하다.
- **할 일**
 - 환경변수 `API_KEY` 또는 `OPENAI_API_KEY`를 설정하고, `engine_core.py`에서 번역을 수행하는 `_translate_en_to_ko` 함수가 실제로 호출되도록 플로우를 수정한다. 예: `collect` 단계 이후 각 기

사에 대해 요약을 생성하고, 영문 기사에는 `_translate_en_to_ko`를 호출해 `summary_ko` 필드를 채운다.

- `selected_articles.json` 및 `selected_keyword_articles.json` 저장 시 `summary_ko` / `summary_en` / `desc` 필드를 포함하도록 `orchestrator.py`를 수정한다.
- 관리자 UI(index.html)에 기사 상세 팝업 또는 편집 영역을 추가하여 기사 요약·번역을 표시하고 필요하면 편집자가 직접 수정할 수 있도록 한다.
- `/api/enrich/keyword`와 `/api/enrich/selection` API는 현재 `summary` 필드만 포함하지만, `summary_ko`를 우선적으로 사용하고 없을 경우 `summary`를 사용하도록 개선한다. 또한 API 호출 결과로 생성된 Markdown 파일을 즉시 UI에 미리보기할 수 있도록 한다.

4.4 백엔드 개선 및 DB 도입 검토

- **왜 필요한가?** 현재 데이터는 JSON 파일(`selected_articles.json` 등)에 저장되어 파일 읽기·쓰기 속도가 느리고 동시성에 취약하다. 또한 기사 상태 변경 시 여러 파일을 병합하는 작업이 필요하여 코드가 복잡하다. 향후 기사 데이터가 많아지면 성능 문제와 데이터 무결성 문제가 발생할 수 있다.
- **할 일**
 - `server_quali.py`와 `engine_core.py`에서 데이터 저장 방식을 추상화하고, PostgreSQL 같은 관계형 DB를 사용할 수 있는 선택지를 제공한다. ORM(Pydantic + SQLAlchemy) 또는 SQLAlchemy를 도입해 Article, Selection, Log 테이블을 정의한다.
 - JSON 저장 방식과 DB 저장 방식을 설정값으로 분기하여, 기존 JSON 방식을 유지하면서도 DB를 쉽게 사용할 수 있도록 한다.
 - DB 이전을 수행할 때는 백업 파일에서 데이터를 읽어 DB에 import하는 마이그레이션 스크립트를 작성해야 한다.

4.5 UX/문서화 보완

- **왜 필요한가?** 현재 보고서 정의서와 실제 UI 사이에 기능 용어 및 동작이 일부 다르고, 외부 사용자(편집자)가 이해하기 어려운 부분이 있다.
- **할 일**
 - 관리자 UI의 게이트 슬라이더와 게이트 통과 조건에 대해 운영자 가이드 문서에 설명을 추가한다. 문서에는 기본값(15개)과 의미, 슬라이더로 조정할 수 있다는 사실을 명시한다.
 - 메뉴와 버튼 이름을 정의서와 일치하도록 다듬는다. 예를 들어 정의서에서는 “보고서 생성”이라고 하지만 UI에는 “Daily 보고서” 등 다른 표현이 있을 수 있다.
 - 반응형 레이아웃을 개선하여 모바일·태블릿에서도 깨짐 없이 표시되도록 CSS를 점검하고, 각 브라우저(Chrome/Edge/Safari)에서 테스트한다.
 - 개발자 및 운영자 가이드북을 최신 코드와 기능에 맞게 업데이트하고, 변경된 API 경로와 파라미터를 반영한다.

5. 맺음말

첨부된 코드와 자료를 분석한 결과, **컬리지널 관리자 시스템은 핵심 기능의 뼈대를 갖추고 있으나 정의서가 요구하는 최종 완료 상태에 도달하려면 도메인 매핑, 자동화, 백업, 요약/번역 파이프라인 등 추가 개발이 필요하다.** 위 작업 지침을 기반으로 보완 개발을 진행하면, 정의서에 규정된 완료 기준을 충족하고 안정적으로 운영 가능한 시스템을 완성할 수 있을 것이다.